

MCTS — Desarrollo de Software Seguro

Reporte de Estado del Frontend

IoT Server — Panel de Administración

IOT-Server Platform

9 de mayo de 2026

Resumen

Este documento describe el estado del frontend del sistema IoT Server, incluyendo funcionalidades operativas, módulos pendientes, problemas conocidos del backend y las actualizaciones sucesivas realizadas entre el 7 y el 9 de mayo de 2026. El stack utilizado es React 18 + TypeScript + Vite + MUI + TanStack Query contra un backend FastAPI + SQLAlchemy + SQLite + JWT (HS256).

Índice

1. Resumen Ejecutivo	3
2. Actualización — Sincronización con Upstream (13 de abril de 2026)	3
2.1. Nuevos módulos incorporados al upstream	3
2.2. Nueva Matriz de Permisos OSO	4
3. Actualización — Mejoras UX y Rastreo de Actividad (13 de abril, fase 2)	4
3.1. Nuevo Sistema de Rastreo de Actividad (ActivityContext)	4
3.2. Dialog de Vista Detalle Genérico (DetalleDialog)	4
4. Actualización — Mejoras de Edición de Estado (13 de abril, fase 3)	5
5. Actualización — Correcciones UX Críticas (14 de abril, fase 4)	5
5.1. Formulario de edición sin datos precargados	5
5.2. Datos sensibles visibles sin protección	5
5.3. Diálogo de cambio de contraseña	5
5.4. Corrección de logout	5
6. Funcionalidades Pendientes por Falta de Endpoints	6
7. Sincronización con Upstream (15 y 20 de abril de 2026)	6

8. Módulo de Roles, OSO y Firma de Peticiones (5 de mayo)	6
8.1. Nuevo CRUD de Roles	6
8.2. Esquema de firma criptográfica (PG/TAG/PF)	6
9. Modo Configuración Inicial (5 de mayo)	7
10.Fase 2 — Configuración RC Completa (5 de mayo)	7
11.Mejoras en Aplicaciones (8 de mayo)	7
12.Restricción de Acceso por Tipo de Cuenta en Login (8 de mayo)	7
13.Habilitación de HTTPS con Certificado Autofirmado (8 de mayo)	8
14.Auto-inyección de Administrador y Paneles de Vinculación (8 de mayo)	8
15.Pruebas de Funcionalidad de la API (9 de mayo)	8
16.Actualización Frontend — Integración Completa (9 de mayo)	9

1. Resumen Ejecutivo

El frontend cubre las operaciones CRUD para las entidades principales (Administradores, Gerentes, Usuarios, Dispositivos, Servicios, Aplicaciones, Roles y Tickets) y la autenticación JWT con verificación de tipo de cuenta. Todas estas funcionalidades están **operativas y probadas** contra el backend en entorno Docker. Se ha implementado firma criptográfica de peticiones (PG/TAG/PF), modo de configuración inicial, HTTPS local, y paneles de vinculación entre entidades.

2. Actualización — Sincronización con Upstream (13 de abril de 2026)

Motivo: El repositorio upstream `Coquita01/IOT-Server` recibió 20+ commits nuevos entre el 7 y el 13 de abril de 2026, introduciendo módulos completos de autorización RBAC (OSO), servicios, aplicaciones, tickets y gestión de sesiones con Valkey. Se sincronizó el fork `GemmaDI300/IOT-Server` y se implementaron todas las actualizaciones correspondientes en el frontend. **El backend no fue modificado.**

2.1. Nuevos módulos incorporados al upstream

Módulo	Ruta	Descripción
Autorización OSO	<code>app/shared/authorization/</code>	Motor de políticas RBAC basado en Open Policy Agent. <code>policies.polar</code> define todos los permisos por rol.
Sesiones Valkey	<code>app/shared/session/</code>	Gestión de sesiones con Redis/Valkey. El backend puede invalidar tokens por servidor.
Servicios	<code>app/domain/service/</code>	CRUD completo. Un servicio pertenece a un administrador.
Aplicaciones	<code>app/domain/application/</code>	CRUD completo con generación automática de <code>api_key</code> .
Tickets de Servicio	<code>app/domain/tickets/</code>	<code>ServiceTicket</code> y <code>EcosystemTicket</code> con guards OSO.

Cuadro 1: Nuevos módulos incorporados al upstream

2.2. Nueva Matriz de Permisos OSO

Recurso	Admin Master	Admin Normal	Gerente	Usuario
Administradores	CRUD	solo lectura	—	—
Gerentes	CRUD	CRUD	solo lectura	—
Usuarios	CRUD	CRUD	crear + editar	editar propio
Dispositivos	CRUD	CRUD	CRUD	solo lectura
Aplicaciones	CRUD	CRUD	solo lectura	—
Servicios	CRUD	CRUD	solo lectura	—
Tickets	CRUD	crear + editar	crear + editar	crear + editar

Cuadro 2: Matriz de permisos OSO

Ningún rol puede eliminar tickets — la política Polar no define la acción delete sobre tickets para ningún rol.

3. Actualización — Mejoras UX y Rastreo de Actividad (13 de abril, fase 2)

3.1. Nuevo Sistema de Rastreo de Actividad (ActivityContext)

Se creó `src/shared/activity/activityContext.tsx` con un React Context que registra cada acción CRUD ejecutada por el usuario en la sesión activa.

Aspecto	Detalle
Almacenamiento	<code>localStorage</code> bajo la clave <code>ïot_activity_log</code> (máx. 50 eventos)
Estructura de un evento	<code>{id, action: "created" "edited" "deleted", entityType, entityName, timestamp}</code>
Persistencia	Sobrevive recargas de página
Provider	<code><ActivityProvider></code> envuelve toda la app en <code>main.tsx</code>
Hook de consumo	<code>useActivity()</code> retorna <code>{events, addEvent, clearEvents}</code>

Cuadro 3: Especificaciones del ActivityContext

3.2. Dialog de Vista Detalle Genérico (DetalleDialog)

Se creó `src/components/DetalleDialog.tsx`, un diálogo de solo lectura que muestra todos los campos de cualquier entidad cuando el usuario hace clic en el botón "Ver" de una fila.

Campo	Renderizado
<code>is_active</code>	Chip verde/rojo (Activo / Inactivo)
<code>priority</code>	Etiqueta en español (Baja / Media / Alta / Crítica)
Fechas (ISO 8601)	<code>toLocaleString()</code> en español
<code>password</code> / <code>password_hash</code>	Oculto — nunca se muestra
UUID / strings	Texto plano
Números	Texto plano

Cuadro 4: Comportamiento de renderizado en `DetalleDialog`

4. Actualización — Mejoras de Edición de Estado (13 de abril, fase 3)

Se permitió modificar el estado (`is_active`) de usuarios, dispositivos y servicios desde el formulario de edición. Se añadió un tipo `"boolean"` a `FieldConfig` para renderizar `Select` con conversión `string↔boolean`, garantizando que el `PATCH` envíe tipos correctos.

5. Actualización — Correcciones UX Críticas (14 de abril, fase 4)

5.1. Formulario de edición sin datos precargados

Problema: `EditarDialog` inicializaba `Formik` con valores vacíos, sobrescribiendo datos válidos al enviar.

Solución: `initialValues` se construyen ahora desde `rowData`. El `PATCH` filtra el payload: omite campos sin cambios y campos vacíos. El campo `password` se omite completamente del formulario de edición.

5.2. Datos sensibles visibles sin protección

Problema: `DetalleDialog` no mostraba campos que el backend oculta por política de seguridad.

Solución: Se definió `SENSITIVE_PERSONAL_FIELDS`. Los campos ausentes se renderizan como `●●●●●●●●` bajo el encabezado *"Datos confidenciales (no devueltos por la API)"*.

5.3. Diálogo de cambio de contraseña

Se creó `CambiarPasswordDialog.tsx` con validación `Yup` (min 8 chars, coincidencia de confirmación). Llama a `PATCH /api/v1/auth/change-password`. Accesible mediante el ícono  en `SidebarLayout.tsx`.

5.4. Corrección de logout

`handleLogout` redirigía a `/login` (ruta inexistente). Se corrigió a `navigate(/login/admin-master)` para que `ProtectedRoute` intercepte correctamente la falta de sesión.

6. Funcionalidades Pendientes por Falta de Endpoints

La asignación de usuarios a servicios está bloqueada porque la relación `User → UserRole → Role → Service` no tiene API expuesta.

Endpoint requerido	Método	Estado backend
<code>/api/v1/user-roles/</code>	GET, POST	× No implementado
<code>/api/v1/services/{id}/users/</code>	GET	× No implementado
<code>/api/v1/manager-services/</code>	GET, POST	× No implementado

Cuadro 5: Endpoints pendientes para asignación de usuarios a servicios

El componente `GestionUsuariosServicioDialog.tsx` ya está diseñado y listo para activarse cuando el backend esté disponible.

7. Sincronización con Upstream (15 y 20 de abril de 2026)

Se integraron commits con:

- Módulo de sesiones (Valkey/Redis)
- Autenticación por puzzle para aplicaciones IoT
- Validación de IP y MAC en dispositivos
- Enum `Priority` en Tickets
- Suites de pruebas backend (29 archivos nuevos)

Impacto en frontend: Ninguno. Todos los cambios son transparentes o manejados por respuestas HTTP 422. Se mantuvo `docker-compose.yml` local con estrategia `-X ours`.

8. Módulo de Roles, OSO y Firma de Peticiones (5 de mayo)

8.1. Nuevo CRUD de Roles

Se añadió `Roles.tsx` con validación Yup (solo letras para `name`). Endpoint: `/api/v1/roles/`. Permisos OSO reflejados en UI: Admin/Manager pueden crear/editar; solo Admin elimina.

8.2. Esquema de firma criptográfica (PG/TAG/PF)

Se implementó `requestSigning.ts` usando Web Crypto API:

- `IDSess = SHA-256(JWT) → hex-64`

- `kEnc = SHA-256(iot-enc-v1:"||JWT) → clave AES-256-CBC`
- `SessApp = SHA-256(iot-sess-v1:"||JWT) → clave HMAC-SHA256`

Genera cabeceras `X-Session-Id`, `X-Request-Pg`, `X-Request-Tag`, `X-Request-Ts`. Degradación graceful si Web Crypto no está disponible. Integrado en todos los hooks de fetch.

9. Modo Configuración Inicial (5 de mayo)

Cuando `VITE_APP_APPLICATION_ID`, `VITE_APP_API_KEY` y `VITE_APP_SERVER_KEY` están vacías, el frontend entra en `SetupModeContext`:

- Sidebar solo muestra *Aplicaciones* + banner amarillo
- Cualquier otra ruta redirige a `/aplicaciones`
- `BackendGate` omite verificación de puzzle hasta configurar credenciales

10. Fase 2 — Configuración RC Completa (5 de mayo)

Se añadió panel visual en `Aplicaciones.tsx` que muestra credenciales copiables y comandos para calcular `VITE_APP_SERVER_KEY`. `docker-compose.yml` inyecta las variables como `ARG` en build time.

Variable	Origen
<code>VITE_APP_APPLICATION_ID</code>	UUID de tabla <code>application</code> en SQLite
<code>VITE_APP_API_KEY</code>	Campo <code>api_key</code> generado por backend
<code>VITE_APP_SERVER_KEY</code>	<code>SHA-256(SECRET_KEY + " puzzle_v1")</code>

Cuadro 6: Variables de configuración RC

11. Mejoras en Aplicaciones (8 de mayo)

- **Logs persistentes tras reinstall:** Zona de desinstalación total con botón *"Limpiar logs de actividad local"* que invoca `clearEvents()`.
- **Credenciales inmediatas:** Prop `onCreateSuccess` en `Gestion.tsx` inyecta apps nuevas al estado local sin recargar.
- **Botones de flujo:** *Insertar en docker-compose.yml* y *Reconstruir frontend* copian bloques YAML y comandos al portapapeles.

12. Restricción de Acceso por Tipo de Cuenta en Login (8 de mayo)

Las 5 pantallas de login ahora usan endpoints separados y validan respuesta:

Pantalla	Endpoint	requiredAccountType	requiredIsMaster
Admin Master	auth-rc/master/login	administrator	true
Admin Normal	auth-rc/admin/login	administrator	false
Gerente	auth-rc/manager/login	manager	—
Usuario (Control)	auth-rc/user/login	user	—
Usuario (Monitoreo)	auth-rc/user/login	user	—

Cuadro 7: Configuración de endpoints y validación por pantalla

Validación en doble capa: Backend filtra por `entity_type` en BD; Frontend verifica `account_type` antes de llamar `setSession()`.

13. Habilitación de HTTPS con Certificado Autofirmado (8 de mayo)

Nginx ahora escucha en `listen 3000 ssl;` con certificado generado en build time (RSA 2048, SHA-256, 825 días, SAN=DNS:localhost,IP:127.0.0.1). Tramo `nginx↔backend` es HTTP interno Docker. `VITE_API_BASE_URL` cambió a URL relativa `/api/v1/` para evitar *mixed content*. CSP actualizada a `'self'` en `connect-src`.

14. Auto-inyección de Administrador y Paneles de Vinculación (8 de mayo)

- `administrator_id` en creación de servicios/aplicaciones es ahora `hidden: true` y se inyecta vía `defaultValues` con el UUID de sesión.
- Nuevo componente reutilizable `LinkPanel.tsx` para gestionar relaciones join-table.
- `Gestion.tsx` recibe prop `renderLinkPanel` que abre diálogo con paneles de vinculación.

Paneles implementados:

Relación	Ubicación	Estado
Servicio ↔ Gerente	Servicios.tsx	✓ Implementado
Servicio ↔ Dispositivo	Servicios.tsx	✓ Implementado
Servicio ↔ Aplicación	Servicios.tsx	✓ Implementado
Rol ↔ Usuario	Roles.tsx	✓ Implementado

Cuadro 8: Estado de paneles de vinculación

15. Pruebas de Funcionalidad de la API (9 de mayo)

Se ejecutaron 49/49 pruebas PASS cubriendo:

- 1 login + 1 logout

- 5 entidades × 5 operaciones CRUD = 25 pruebas
- 4 relaciones × 3 operaciones (vincular/listar/desvincular) = 12 pruebas
- Cleanup y listados extras

Correcciones aplicadas al backend tras merge upstream: `model.py` (bloque huérfano, `AuditLog`), `config.py` (`SESSION_TTL_SECONDS`), `rate_limit.py` (implementación), `main.py` (imports, duplicados), `oso_config.py` (registro duplicado), `pyproject.toml` (`loguru`).

16. Actualización Frontend — Integración Completa (9 de mayo)

- `validation.ts`: `model` y `mac` en dispositivos pasaron a opcionales, alineados con Pydantic backend.
- `Servicios.tsx`: Añadido `LinkPanel` para aplicaciones.
- `Aplicaciones.tsx`: `administrator_id` oculto correctamente.
- `Gerentes.tsx`: Añadidas props `showDetail={true}` y `entityTypeLabel="Gerente"`.

Módulo	CRUD	Auto-inyectado	Detalle	Vinculaciones
Administradores	✓	—	✓	—
Gerentes	✓	—	✓	—
Usuarios	✓	—	✓	—
Dispositivos	✓	—	✓	—
Aplicaciones	✓	<code>administrator_id</code>	—	—
Servicios	✓	<code>administrator_id</code>	—	Gerentes, Dispositivos, Aplicaciones
Roles	✓	—	—	Usuarios

Cuadro 9: Estado final de módulos tras integración completa

Nota de despliegue: Para producción, reemplace el certificado autofirmado por uno de CA reconocida (Let's Encrypt) y asegure también el tramo interno `nginx↔backend` con TLS mutuo o red privada aislada.